Übungsblatt 7

Willkommen zum Praktikum zu Algorithmen und Datenstrukturen (ADS).

Aufgabe 1. Die Schlange mit einem Feld in der Vorlesung war begrenzt. Schreiben Sie eine Version `flex_queue` der Schlange, die falls die Schlange voll wird, neuen Speicher allokiert. Realisieren Sie das in den Dateien `flexqueue.h` und `flexqueue.cpp`. Nutzen Sie Prototypen für die Funktionen von `flex_queue` im Header. Wir vergrößern die Kapazität der Schlange um das doppelte. Gleichzeitig soll die Kapazität der Schlange wieder sinken, wenn die Auslastung der Schlange unter 25 Prozent der Kapazität sinkt und zwar um die Hälfte. Wir beginnen mit einer Kapazität von 8 und werden nie weniger als 8 Elemente haben.

Testen Sie Ihre Implementierung zunächst selbst in der Datei `fqmain.cpp` indem Sie 1.000 Elemente einfügen und dann wieder entfernen. Es müssen die gleichen Elemente in der gleichen Reihenfolge vorkommen. Überprüfen Sie dabei die geänderten Kapazitäten und stellen Sie sicher, dass es keine Speicherprobleme gibt. Testen Sie Ihre Implementierung anschließend mittels des Unit-Tests in `test_flexqueue.cpp`.

Aufgabe 2. Nutzen Sie die Warteschlange der vorangehenden Aufgabe, um das folgende Szenario zu realisieren: n Kinder stehen im Kreis und zählen reihum einen Abzählreim mit k Silben. Wer die letzte Silbe abbekommt, scheidet aus. Dann wird neu gezählt, bis ein Kind übrig bleibt.

Realisieren Sie `int abzaehlreim(int n, int k)`, die zurückgibt, welches Kind (von 0 bis `n-1`) gewinnt. Nutzen Sie dazu eine Warteschlange. Entnehmen Sie das vorderste Kind. Fügen Sie es $k-1$ mal hinten wieder ein, beim k-ten Mal scheidet es aus.

Testen Sie mit $n = 7, k = 3$, dabei muss 3 zurückgegeben werden. Testen Sie auch mit $n = 10, k = 2$, dabei müssten Sie 4 erhalten

Aufgabe 3. Realisieren Sie eine Warteschlange für `string` als einfach verkettete Liste mit eigenem Knoten-Struct in einem `struct linked_queue`. Verwenden Sie einen `first`- und einen `last`-Pointer. Sie verwalten die Daten im Freispeicher selbst.

Überlegen Sie, an welchem Ende Sie einfügen und an welchem Sie entfernen, so dass beide Operationen in $O(1)$ möglich sind.

- `void lqueue_enter(linked_queue* lq, string s)`: Element einfügen
- `string lqueue_leave(linked_queue* lq)`: Element am anderen Ende entfernen
- `bool lqueue_is_empty(linked_queue* lq)`: Prüfen, ob die Warteschlange leer ist
- `size_t lqueue_size(linked_queue* lq)`: Bestimmen der Anzahl der Elemente in der Warteschlange

Realisieren Sie die Funktionalität in `linkedqueue.h` und `linkedqueue.cpp`. Testen Sie Ihre Implementierung mit `test_linkedqueue.cpp`.

Aufgabe 4. Wir verwenden die Warteschlange aus der letzten Aufgabe, um Wörter zu sortieren. Sie verwenden 26 Warteschlangen, eine für jeden Buchstaben a–z.

Gegeben ist ein `vector<string>` mit Wörtern derselben Länge d. In jeder Runde verteilen Sie die Wörter nach dem aktuellen Zeichen in die passende Schlange, beginnend mit dem letzten Zeichen (Position $d-1$), dann dem vorletzten, bis zum ersten. Nach jeder Runde sammeln Sie die Wörter der Reihe nach wieder ein (erst Schlange a, dann b, ..., dann z). Nach d Runden sind die Wörter alphabetisch sortiert.

Lesen Sie die Wörter aus `atwi80d.txt` mit `read_words` ein. Filtern Sie alle Wörter einer festen Länge heraus, zum Beispiel alle Wörter der Länge 5. Konvertieren Sie alle Buchstaben zu Kleinbuchstaben und verwerfen Sie Wörter mit Sonderzeichen oder Ziffern.

Sortieren Sie die übrigen Wörter mit Ihrem Verfahren und messen sie die Laufzeit. Was ist die Komplexität dieses Sortierverfahrens?